电子科技大学
University of Electronic Science and Technology of China

第6章 全局优化
（启发式算法）

2022/11/21

全局优划

- 很多优化问题并不能利用梯度法、牛顿法等找到全局最优解。如果目标函数最优性很重要，我们怎么办?
- 现实应用中的有些复杂问题的目标函数并不一定能求导（梯度），甚至无法显式表达出来。我们如何设计算法?

本章讨论的算法不需要对优化问题的结构进行限制，更加通用。

随机搜索

- 随机搜索算法也叫概率搜索方法。算法会在可行集中随机采样，逐步完成搜索。

$$\min f(\boldsymbol{x})$$
$$s.t. \boldsymbol{x} \in \Omega$$

- 基本思想:随机选定一个初始点$\boldsymbol{x}^0 \in \Omega$，从一个$\boldsymbol{x}^0$的邻域集合$N(\boldsymbol{x}) \subset \Omega$中随机选择一个点，作为下一迭代的备选点。

- 邻域$N(\boldsymbol{x})$：一般指的是离$\boldsymbol{x}$比较“接近”的点。 可使用均匀分布或高斯分布作为$N(\boldsymbol{x})$的分布函数。

随机搜索

随机搜索的步骤:

1. 令$k=0$,选择初始点$x^{(0)} \in \Omega$
2. 从$N\left(\boldsymbol{x}^{(k)}\right)$中选出一个备选点$\boldsymbol{z}^{(k)}$
3. 如果$f\left(\boldsymbol{z}^{(k)}\right) \leq f\left(\boldsymbol{x}^{(k)}\right)$,$\boldsymbol{x}^{(k+1)}=\boldsymbol{z}^{(k)}$，否则$\boldsymbol{x}^{(k+1)}=\boldsymbol{x}^{(k)}$
4. 如果满足规则，停止迭代，
5. 令$k=k+1$，返回第2步。

停止规则一般设定迭代次数、运行时间或者目标函数的改善程度。

选取一个$N\left(x^{(k+1)}\right)$可以看成$\boldsymbol{x}^{(k+1)}=\boldsymbol{x}^{(k)}+\boldsymbol{d}^{(k)}$的形式，$\boldsymbol{d}^{(k)}$是随机产生的，可能是0。那么随机搜索也类似下降搜索。

随机搜索

如果$\boldsymbol{x}^{(0)}$是一个局部极小点，$N(\boldsymbol{x}^{(0)})$足够小，使得其中的点对应的目标函数值不会比$\boldsymbol{x}^{(0)}$对应的目标函数值小。这时，随机搜索算法就会卡住。

- 解决方法1:设置更大的$N(\boldsymbol{x}^{(0)})$，可以从空间上来想象，如果邻域解允许的范围非常广，备选点太多了，错误的备选点可能导致算法收敛非常缓慢。
- 解决方法2：对随机搜索进行修改，允许在两次连续的迭代中，算法产生的新点比当前点目标函数值差，以此走出局部最优。

模拟退火算法也就是属于第二种解决方式

模拟退火

模拟退火算法

1. 令$k=0$,选择初始点$x^{(0)} \in \Omega$
2. 从$N\left(x^{(k)}\right)$中选出一个备选点$z^{(k)}$
3. 以概率$p\left(k, f\left(z^{(k)}\right), f\left(x^{(k)}\right)\right)$ 执行$x^{(k+1)}=z^{(k)}$,否则执行$x^{(k+1)}=x^{(k)}$
4. 如果满足规则，停止迭代
5. 令$k=k+1$，返回第2步

模拟退火

一般来说可设置

$$p(k, f(z^{(k)}), f(x^{(k)}))$$
$$= \min\left(1, \exp\left(-\frac{f(z^{(k)}) - f(x^{(k)})}{T_k}\right)\right)$$

T_k是一系列单调递减正数序列，也叫做温度参数。（这种形式的接受概率由Boltzmann提出）

1. 如果$f(z^{(k)}) \leq f(x^{(k)})$,$x^{(k+1)} = z^{(k)}$
2. 如果$f(z^{(k)}) > f(x^{(k)})$,以概率$\exp\left(-\frac{f(z^{(k)})-f(x^{(k)})}{T_k}\right)$执行$x^{(k+1)} = z^{(k)}$,否则执行$x^{(k+1)} = x^{(k)}$

模拟退火

- 由于可能有一定概率接受一个目标值比当前解要差的解，因此算法停止时的当前解$x^{(k)}$并不是搜索到的最好解，我们需要一个x_{best}来保持找到的最好解。

模拟退火

模拟退火算法(详细版)：

1. 令$k=0$,选择初始点$x^{(0)} \in \Omega, x_{best}=x^{(0)}$
2. 从$N\left(x^{(k)}\right)$中选出一个备选点$z^{(k)}$
3. 如果$f\left(z^{(k)}\right) \leq f\left(x^{(k)}\right), x^{(k+1)}=z^{(k)}$
4. 如果$f\left(z^{(k)}\right) > f\left(x^{(k)}\right)$,以概率$\exp\left(-\frac{f\left(z^{(k)}\right)-f\left(x^{(k)}\right)}{T_k}\right)$执行$x^{(k+1)}=z^{(k)}$,否则执行$x^{(k+1)}=x^{(k)}$
5. 如果$f\left(z^{(k)}\right) \leq f\left(x_{best}\right), x_{best}=z^{(k)}$
6. 如果满足规则，停止迭代，否则$k=k+1$，返回第2步

模拟退火

模拟退火算法特点：

- 接受概率函数$\exp\left(-\frac{f\left(z^{(k)}\right)-f\left(x^{(k)}\right)}{T_k}\right)$随着$T_k$减小而逐渐减小到0。也就是说，随着算法迭代次数的增加，算法能接受更差解的概率越来越小了。
- 模拟退火算法一开始希望在整个可行解内进行搜索，随着迭代的进行，搜索的范围逐渐集中到目标函数效果较好的区域搜索。
- 模拟退火源于冶金过程的启发。在炼铁等冶金行业中，人们会先将金属加热到一定程度，此时由于温度较高，原子开始变得活跃，内能比较高。随着金属的冷却，原子逐渐变得有序，内能减小。如果冷却过程足够慢，那么可以保证最终的内能低于开始阶段的内能，这样可以改善金属的晶体结构。

模拟退火

- 对于如何设计温度参数T_k的值呢?

一般而言，温度最好以较慢的速度降低。Hajek给出了如下的一种设计建议：

$$T_k = \frac{\gamma}{\log(k+2)}$$

其中$\gamma > 0$为常数，一般需要足够大，以保证算法能够跳出局部极小点附近的区域。

- 由于模拟退火使用度广，容易操作，关于其研究非常广泛。并且，在组合优化领域也大量使用模拟退火算法求解难问题。如我们开篇介绍的旅行商问题，就可以利用模拟退火来解决。

模拟退火

- 理论上来说，模拟退火在足够多的迭代次数下是可收敛。但是由于理论迭代时长太大，甚至可能超越精确搜索所需要的时间，所以该理论结果并不能指导实际操作。

Simulated Annealing: A Proof of Convergence

Vincent Granville, Mirko Křivánek, and Jean-Paul Rasson

Abstract—We prove the convergence of the simulated annealing procedure when the decision to change the current configuration is blind of the cost of the new configuration. In case of filtering binary images, the proof easily generalizes to other procedures, including that of Metropolis. We show that a function Q associated with the algorithm must be chosen as large as possible to provide a fast rate of convergence. The worst case (Q constant) is associated with the "blind" algorithm. On the other hand, an appropriate Q taking sufficiently high values yields a better rate of convergence than that of Metropolis procedure.

粒子群优化算法

- 粒子群算法由James Kennedy and Russell C. Eberhart 1995年提出。

- 粒子群算法会维护一组迭代点，称为群。群中的每个点被视为一个粒子。每个粒子都有一个带有随机性的速度向量。算法每一次迭代会按该速度向量移动，目的是使得粒子最终聚集到最优点附近。

- 这种想法主要源于模仿自然界鱼群、鸟群和蜂群等群体的形成过程。

粒子群优化算法

粒子在$\mathbb{R}^n$中搜索的基本思想：

- 在$\mathbb{R}^n$中随机找到产生一组数据点，为每个点赋予一个速度，构成了一个速度向量。这些点视为粒子，并以一定速度移动。接下来，针对每个数据点计算对应的目标函数值，然后基于计算结果产生一组新的数据点，赋予新的速度并移动。
- 移动过程中，每个粒子的最好值记为$pbest$，全局最好值记为$gbest$。在每次迭代中，每个粒子根据$pbest$和$gbest$的值调整自己的运动速度，以此实现粒子间“交互”。

粒子群优化算法

在基本的粒子群优化算法中，一个迭代点称为一个粒子，迭代点保存的解称为该粒子的位置。

- $d \in \mathbb{Z}$:群体中粒子的个数。
- $\boldsymbol{x}_i \in \mathbb{R}^n$：第i个粒子的位置（也就是第i个解）。
- $\boldsymbol{p}_i \in \mathbb{R}^n$：第i个粒子迭代过程中最好位置。
- $\boldsymbol{g} \in \mathbb{R}^n$：整个搜索过程中找到的最好位置。
- $A \circ B$：矩阵A和B的schur积，要求A和B是同维矩阵。Schur积就是点乘，运算结果是一个和A或B维度相同的矩阵，每一个元素是A和B相同位置的元素乘积。

粒子群优化算法

该算法要求预先设定ω, c_1, c_2三个参数。算法的步骤如下；

1. 令$k=0$。随机产生一个初始的粒子群，即产生d个粒子的位置$x_i^{(0)}$及其对应的速度$\boldsymbol{v}_i^{(0)}$。对于每一个粒子$i=1,\dots,d$，设置

$$\boldsymbol{p}_i^{(0)} = \boldsymbol{x}_i^{(0)}; \boldsymbol{g}^{(0)} = argmin_{\boldsymbol{x}\in\left\{\boldsymbol{x}_1^{(0)},\dots,\boldsymbol{x}_d^{(0)}\right\}} f(\boldsymbol{x})$$

2. 第k次迭代，针对每个粒子i （$i=1,\dots d$），随机从(0，1)区间产生n个数，构成n维向量$\boldsymbol{r}_i^{(k)}$(该向量表示个体最好位置在更新速度时的权重)；同样的方法再随机构造$\boldsymbol{s}_i^{(k)}$（该向量表示整体最好位置在更新速度时的权重），计算速度

$$\boldsymbol{v}_i^{(k+1)} = \omega\boldsymbol{v}_i^{(k)} + c_1\boldsymbol{r}_i^{(k)} \circ \left(\boldsymbol{p}_i^{(k)} - \boldsymbol{x}_i^{(k)}\right) + c_2\boldsymbol{s}_i^{(k)} \circ (\boldsymbol{g}^{(\boldsymbol{k})} - \boldsymbol{x}_i^{(k)})$$

然后更新粒子i的位置

$$\boldsymbol{x}_i^{(k+1)} = \boldsymbol{x}_i^{(k)} + \boldsymbol{v}_i^{(k+1)}$$

粒子群优化算法

3. 更新每个粒子的最好位置$\boldsymbol{p}_i$：针对每个$i=1,\dots d$,如果$f\left(\boldsymbol{x}_i^{(k+1)}\right)<f(\boldsymbol{p}_i^{(k)})$,跟新 $\boldsymbol{p}_i^{(k+1)}=\boldsymbol{x}_i^{(k+1)}$，否则$\boldsymbol{p}_i^{(k+1)}=\boldsymbol{p}_i^{(k)}$

4. 更新群体的最好$\boldsymbol{g}^{(k)}$：如果

$$min_{\boldsymbol{x}\in\left\{\boldsymbol{x}_1^{(k+1)},\dots,\boldsymbol{x}_d^{(k+1)}\right\}}f(\boldsymbol{x})<f\left(\boldsymbol{g}^{(k)}\right),$$

则 $\boldsymbol{g}^{(k+1)}=argmin_{\boldsymbol{x}\in\left\{\boldsymbol{x}_1^{(k+1)},\dots,\boldsymbol{x}_d^{(k+1)}\right\}}f(\boldsymbol{x})$,。

否则，令$\boldsymbol{g}^{(k+1)}=\boldsymbol{g}^{(k)}$

5. 如果满足停止条件，则停止迭代。否则令$k=k+1$，返回第2步

以上算法中，参数ω表示惯性参数，一般取小于1的值。参数c_1和c_2决定了粒子趋向于“好位置”的程度，一般建议$c_1=c_2\approx 2$

粒子群优化算法

粒子群变种：

- Clec提出了收敛因子粒子群算法，速度更新公式为:

$$\boldsymbol{v}_i^{(k+1)} = \kappa\left(\boldsymbol{v}_i^{(k)} + c_1\boldsymbol{r}_i^{(k)} \circ \left(\boldsymbol{p}_i^{(k)} - \boldsymbol{x}_i^{(k)}\right) + c_2\boldsymbol{s}_i^{(k)} \circ \left(\boldsymbol{g}^{(\boldsymbol{k})} - \boldsymbol{x}_i^{(k)}\right)\right)$$

收敛系数$\kappa = \frac{2}{|2-\phi-\sqrt{\phi^2-4\phi}|}$，其中$\phi = c_1 + c_2$, $\phi > 4$

如ϕ=4.1时， $\kappa = 0.729$

在实际收敛中还给速度指定了一个上限v_{max}

遗传算法

- 类似于粒子群算法，遗传算法也是受到生物遗传理论的启发而来的。John Holland在20世纪60年代末70年代初提出。在接下来的讲述中，为了描述方便，我们以最大化问题为优化目标。

$$\max f(x)$$
$$\text{s.t.}\ x \in \Omega$$

最小化问题目标取负→最大化问题

遗传算法

遗传算法是在对解进行重新编码后进行的，因此有以下概念。

- Ω的映射为一个字符串的集合，这些字符串是等长的，也称为**染色体**。
- 每个染色体由**字符表**元素排列得到。如常见字符表{0,1}，染色体就是二进制字符串。
- L表示**染色体的长度**，即字符串字符数量。
- 染色体对应的目标函数值称为适应度。用$\boldsymbol{x}$表示染色体，适应度就是$f(\boldsymbol{x})$。一般假设f是非负函数。

遗传算法

遗传算法也是一个种群算法，其大致过程为：

1.令$k = 0$,产生一个初始种群$P(0)$
2.评估$P(k)$,即计算$P(k)$每个个体的适应度。
3.如果满足停止规则，则停止迭代
4.从$P(k)$中**选择**新的种群$M(k)$
5.**进化**$M(k)$,构造新种群$P(k+1)$
6.令$k = k + 1$,回到第二步。

遗传算法

遗传算法选择过程：

在第k次迭代中，遗传算法从当前的种群$P(k)$构造出一个新的种群$M(k)$，大小和原来种群一致。$M(k)$也称为配对池，是在$P(k)$的基础上进行随机构造得到。构造方法一般是采用**轮盘赌模式**或则**锦标赛模式**。

- **轮盘赌模式**：执行N次轮盘旋转操作，每次旋转会按照染色体适应度的大小概率选取一个染色体，然后将染色体放入$M(k)$
- **锦标赛模式**：随机选取两个染色体，比较他们的适应度，将好的放入$M(k)$。执行N次。

遗传算法

- **交叉：**构造好配对池$M(k)$，下一步就是要进行交叉。交叉类似于生物的繁殖，即从$M(k)$中随机抽出两个解来，通过交换一部分信息，构造出两个新的染色体。然后将这两个新的染色体放回。

交叉也有随机性，通常定义一个概率参数p_c，每个染色体以p_c概率被选中参与交叉。

遗传算法

单点交叉：

染色体长度$L = 6$，编码采用二进制符号表{0,1},一对父代染色体为000000和111111，交叉点为4，对其进行交叉后，可得子代染色体000011和111100

多点交叉：

染色体长度$L = 9$，编码采用二进制符号表{0,1},一对父代染色体为000000000和111111111，交叉点为3和7，对其进行交叉后，可得子代染色体000111100和111000011

遗传算法

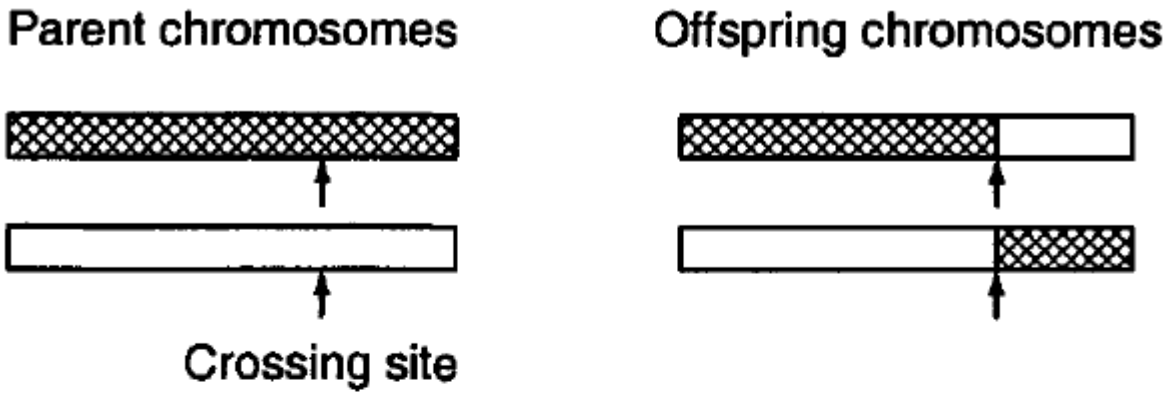


Figure 14.1 Illustration of basic crossover operation

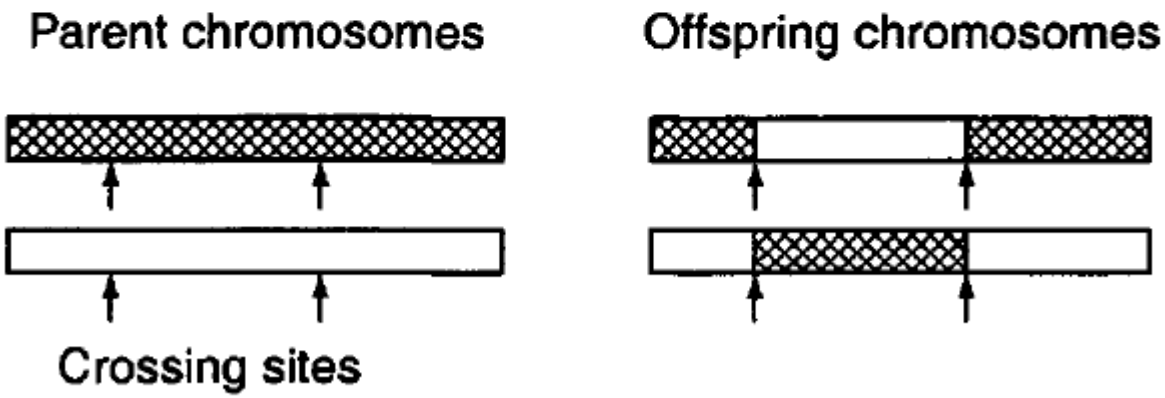


Figure 14.2 Illustration of two-point crossover operation

遗传算法

完成交叉操作之后，利用子代染色体替代配对池中一个的父代染色体。这样，就实现了对配对池的修改，并保持了原来的容量。接下来，需要进行变异操作。

- **变异**：从配对池中逐一取出染色体，以变异概率P_m随机改变其中的字符。例如在二进制编码中，字符$0 \to 1$或$1 \to 0$。通常情况下，变异的概率很小，只有很少的染色体才会经历变异操作。这些变异了的染色体中，也是只有很少的字符被改变。相对于交叉操作，变异操作在遗传算法的影响比较小。
- **更新种群：**通过对$M(k)$一系列的操作，我们得到了新的种群$P(k+1) = M(k)$

遗传算法

- **遗传算法的步骤总结：**

1.令$k=0$,产生一个初始种群$P(0)$

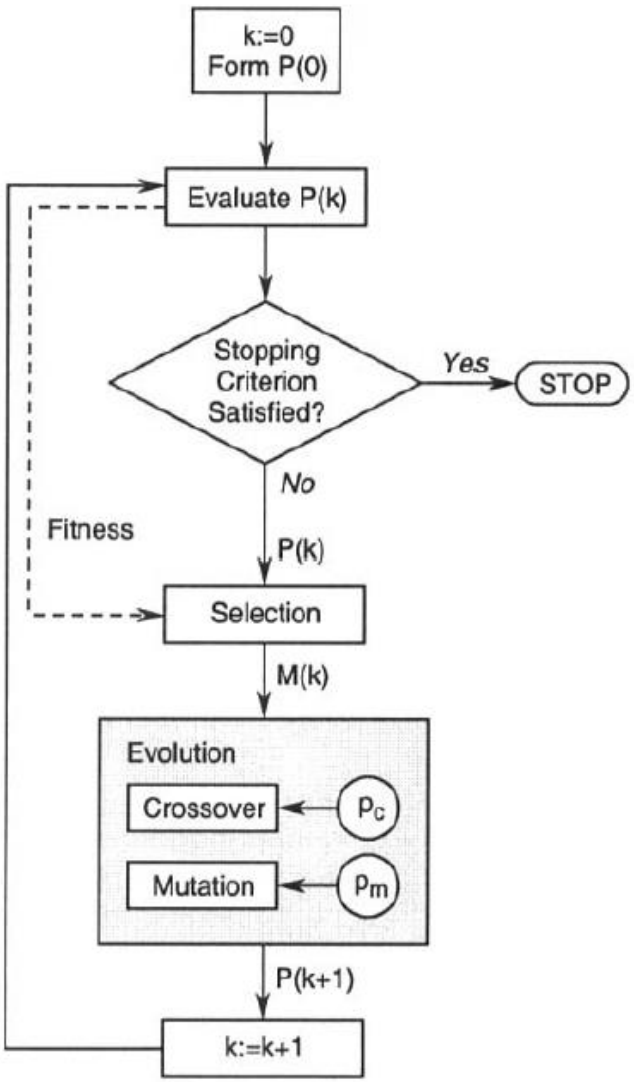
2.评估$P(k)$,即计算$P(k)$每个个体的适应度.

3.如果满足停止规则，则停止迭代

4.从$P(k)$中选择$M(k)$

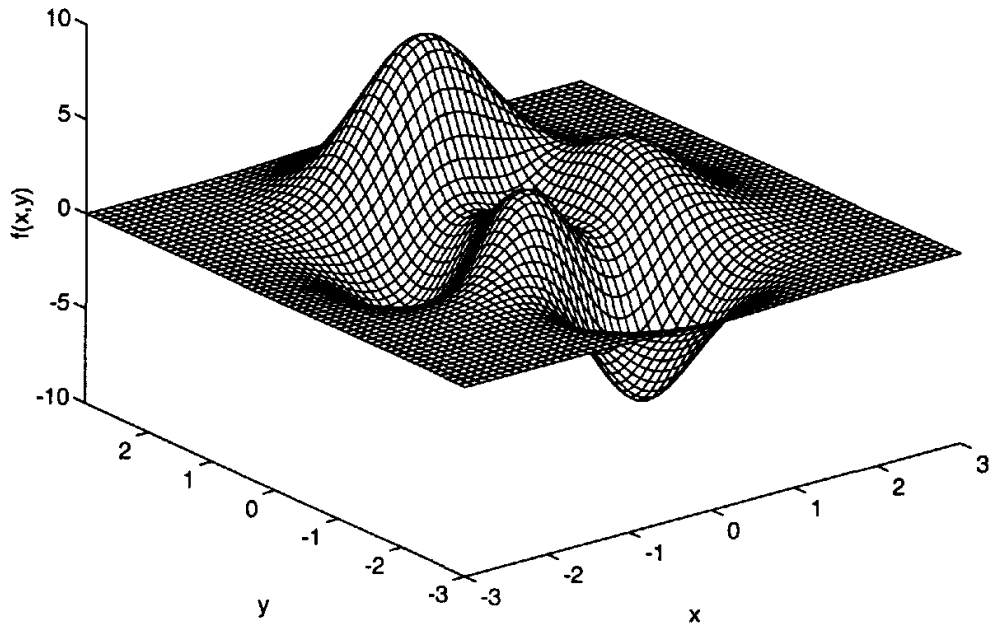
5.进化$M(k)$,构造新种群$P(k+1)$

6.令$k=k+1$,回到第二步



遗传算法举例

计算peaks函数：$f(x,y)=3(1-x)^2e^{-x^2-(y+1)}-10\left(\frac{x}{5}-x^3-y^5\right)-\frac{e^{-(x+1)^2-y^2}}{3}$在$\Omega=\{[x\ y]^T:-3\le x,y\le 3\}$上的极大点。



遗传算法举例

- 设定$\mathrm{L} = 32$,前16位表示x，后16位表示y。
- 将$[-3,3]$区间映射到$[0,2^{16}-1]$区间，那么该$[-3,3]$任何一个数可以采用16位二进制表示。

如$[x\ y]^T = [-1\ 3]^T$可表示为

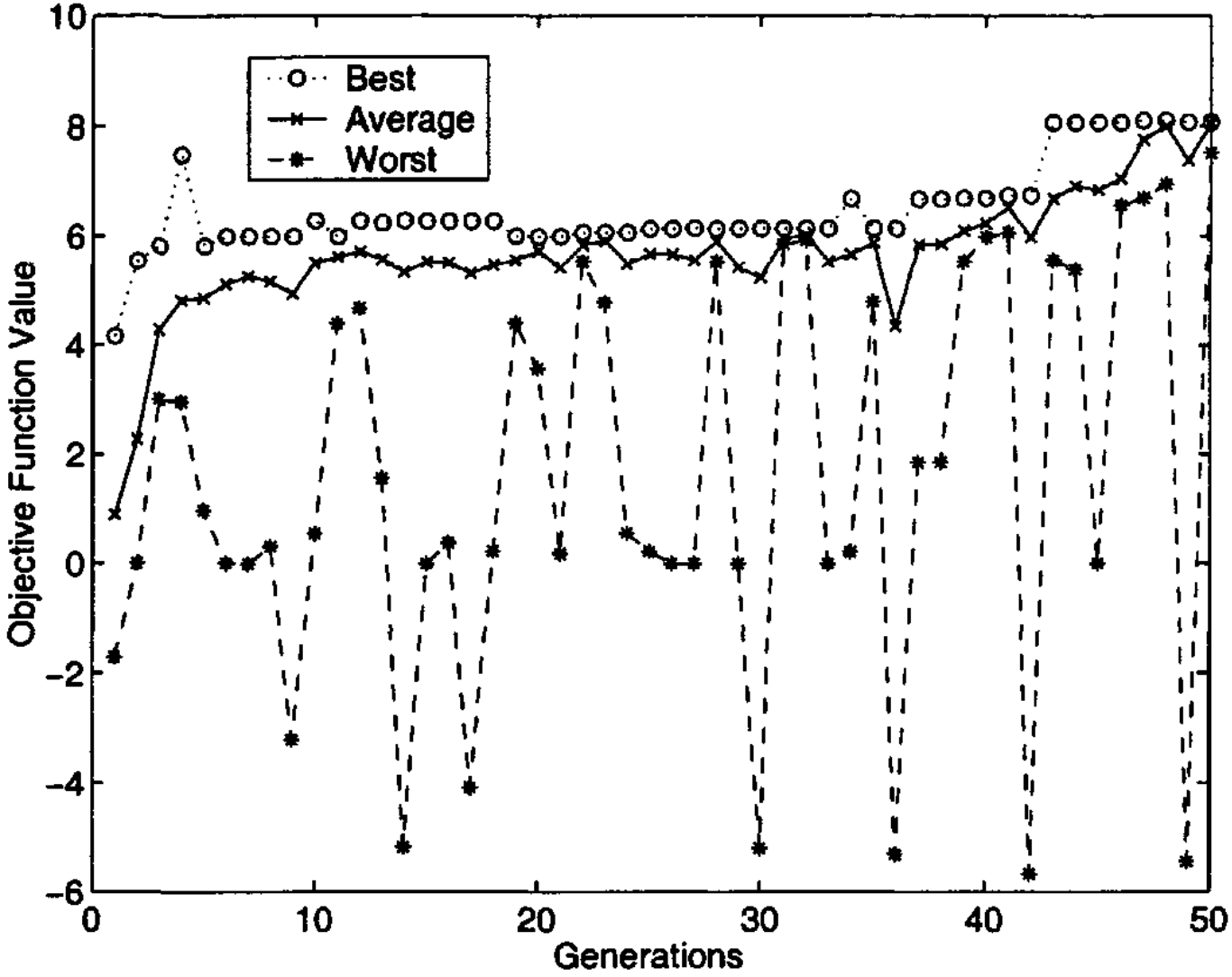
$$0101010101010101111111111111111$$

红色表示x,绿色表示y

- 设定种群容量20，开展50次迭代，交叉概率$p = 0.75$，变异概率$p_m = 0.0075$.
- 迭代50次得到最好点$[0.0615,1.5827]^T$，$f(x^*) = 8.1013$

(实际最优解为$[-0.0093,1.5814]^T$,值为8.1062)

遗传算法举例


Best
Average
Worst
Objective Function Value
Generations
10
8
6
4
2
0
-2
-4
-6
0
10
20
30
40
50

实数编码遗传算法

- 对于$\max f(x)\ \mathrm{s.t.}\ \mathrm{x} \in \Omega \subseteq \mathbb{R}^n$，二进制表示的遗传算法实际上是在优化复合函数：

$$\max f(g(x))$$
$$s.t.\ x \in \{y \in \{0,1\}^L : g(y) \in \Omega\}$$

其中$g(x): \{0,1\}^L \to \Omega$表示二进制的解码函数。如果$y$是一个二进制染色体，$g(y)$就是约束集$\Omega$中的一个点。

但是，复合函数问题将会比原问题更为复杂。是否能直接在原函数上进行求解操作，并直接使用实数$\mathbb{R}^n$编码？

实数编码遗传算法

- 交叉方式(设父代为$\boldsymbol{x}, \boldsymbol{y}$):

1. 求平均值。$\boldsymbol{z} = \frac{\boldsymbol{x}+\boldsymbol{y}}{2}$，用$z$替代$x$或$y$。
2. 加扰动的平均值。$\boldsymbol{z_1} = \frac{\boldsymbol{x}+\boldsymbol{y}}{2} + \boldsymbol{w}_1, \boldsymbol{z_2} = \frac{\boldsymbol{x}+\boldsymbol{y}}{2} + w_2, w_1, w_2$是随机向量，均值为0。如果$z \notin \Omega$，进行投影处理。
3. 构造凸组合。$\boldsymbol{z}_1 = \alpha\boldsymbol{x} + (1-\alpha)\boldsymbol{y}, \boldsymbol{z}_2 = (1-\alpha)\boldsymbol{x} + \alpha\boldsymbol{y}$，$\alpha \in (0,1)$。
4. 加扰动的凸组合。$\boldsymbol{z}_1 = \alpha\boldsymbol{x} + (1-\alpha)\boldsymbol{y} + \boldsymbol{w}_1, \boldsymbol{z}_1 = (1-\alpha)\boldsymbol{x} + \alpha\boldsymbol{y} + \boldsymbol{w}_2$，$\alpha \in (0,1), \boldsymbol{w}_1, \boldsymbol{w}_2$是随机向量，均值为0。

注意：如果$z \notin \Omega$，进行投影处理。

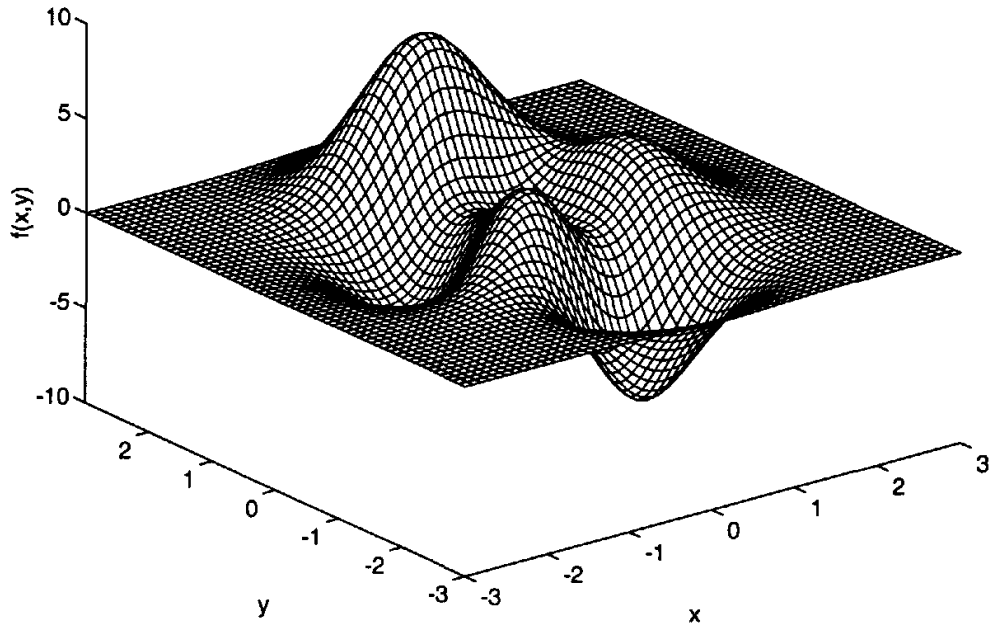
实数编码遗传算法

- 变异操作：

1. $x' = x + w$,是随机向量，均值为0。如果$x' \notin \Omega$，进行投影处理。
2. x和约束集内任何一点的凸组合作为变异后的染色体。$\alpha \in (0,1), w \in \Omega$任意一点,计算$x' = \alpha x + (1 - \alpha)w$。当$\Omega$是凸集时，变异后的染色体一定位于约束集内。

实数编码遗传算法

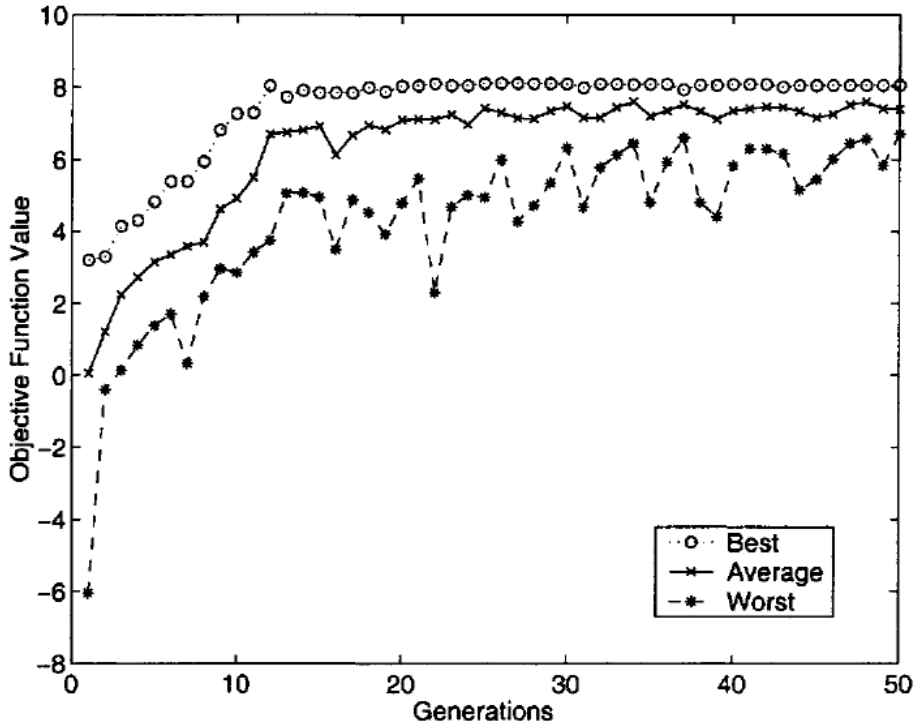
- 计算peaks函数：$f(x,y)=3(1-x)^2e^{-x^2-(y+1)}-10\left(\frac{x}{5}-x^3-y^5\right)-\frac{e^{-(x+1)^2-y^2}}{3}$在$\Omega=\{[x\ y]^T:-3\le x,y\le 3\}$上的极大点。（采用实数编码、交叉方式4、变异方式2）



实数编码遗传算法

- 设定种群容量20，开展50次迭代，交叉概率$p_c = 0.75$，变异概率$p_m = 0.0075$。
- 最好点$[-0.0096\ 1.5845]^T$，$f(x^*) = 8.1061$

(实际最优解为$[-0.0093, 1.5814]^T$,值为8.1062)



传统 VS 遗传优化

数学优化vs进化计算
0
数学优化是解决优化问题最强大最有效的工具
醒醒吧，数学优化只会玩数学，还有什么比进化计算更简单更好用？
1
你们用过对方的方法吗？
我为什么要浪费时间去学垃圾？
2
真难想象解一个凸优化也用GA去做。
声称能找到全局最优，不过是仅仅在几个测试函数上跑跑罢了，速度还慢的要死。
一个好的研究者怎么能不用凸优化呢，再不济线性规划也可以啊。
3
知道problem-independent吗？给我一个cost function 我就能撬动整个地球，你行吗？
知乎 @文雨之

小结

- 朴素的随机搜索：（单点迭代）从邻域中随机取一个点。当该点拥有更好目标函数值时，接受该点。
- 模拟退火算法：（单点迭代）从邻域中以概率接受一个点，接受差的解的概率随着时间而减小（降温过程)。
- 粒子群算法：（多点迭代）维护一个粒子群，每一个粒子拥有自己的速度。每一步迭代所有的粒子移动一次，并更新自己的速度。
- 遗传算法：

1.维护初始种群,即一堆染色体，每个染色体对应一个解。

2.轮盘赌或则锦标赛构建一个配对池。

3.随机选择父代进行交叉，构造出子代，并利用子代替换配对池的父代染色体。

4.以小概率对子代染色体进行变异。

5.利用变异后的子代种群更新当前的种群，继续2-4操作。

- 本章算法都带有随机性。

谢 谢
THANKS